JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface PrimitiveIterator.OfDouble

All Superinterfaces:

Iterator<Double>, PrimitiveIterator<Double, DoubleConsumer>

Enclosing interface:

PrimitiveIterator<T,T_CONS>

```
public static interface PrimitiveIterator.OfDouble
extends PrimitiveIterator<Double, DoubleConsumer>
```

An Iterator specialized for `double` values.

Since:

1.8

Nested Class Summary

Nested classes/interfaces declared in interface PrimitiveIterator
PrimitiveIterator.OfDouble, PrimitiveIterator.OfInt, PrimitiveIterator.OfLong

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
default void	forEachRemaining(Consumer<? super Double> action)	Performs the given action for each remaining element until all elements have been processed or the action throws an exception.
default void	forEachRemaining(DoubleConsumer action)	Performs the given action for each remaining element until all elements have been processed or the action throws an exception.
default Double	next()	Returns the next element in the iteration.

double	**nextDouble**()	Returns the next double element in the iteration.

Methods declared in interface Iterator

hasNext, remove

Method Details

nextDouble

```
double nextDouble()
```

Returns the next double element in the iteration.

Returns:

the next double element in the iteration

Throws:

NoSuchElementException - if the iteration has no more elements

forEachRemaining

```
default void forEachRemaining(DoubleConsumer action)
```

Performs the given action for each remaining element until all elements have been processed or the action throws an exception. Actions are performed in the order of iteration, if that order is specified. Exceptions thrown by the action are relayed to the caller.

The behavior of an iterator is unspecified if the action modifies the source of elements in any way (even by calling the remove method or other mutator methods of Iterator subtypes), unless an overriding class has specified a concurrent modification policy.

Subsequent behavior of an iterator is unspecified if the action throws an exception.

Specified by:

forEachRemaining in interface PrimitiveIterator<Double, DoubleConsumer>

Implementation Requirements:

The default implementation behaves as if:

```
    while (hasNext())
        action.accept(nextDouble());
```

Parameters:

`action` - The action to be performed for each element

next

`default Double next()`

Returns the next element in the iteration.

Specified by:

`next` in interface `Iterator<Double>`

Implementation Requirements:

The default implementation boxes the result of calling `nextDouble()`, and returns that boxed result.

Returns:

the next element in the iteration

forEachRemaining

`default void forEachRemaining(Consumer<? super Double> action)`

Performs the given action for each remaining element until all elements have been processed or the action throws an exception. Actions are performed in the order of iteration, if that order is specified. Exceptions thrown by the action are relayed to the caller.

The behavior of an iterator is unspecified if the action modifies the collection in any way (even by calling the `remove` method or other mutator methods of `Iterator` subtypes), unless an overriding class has specified a concurrent modification policy.

Subsequent behavior of an iterator is unspecified if the action throws an exception.

Specified by:

`forEachRemaining` in interface `Iterator<Double>`

Implementation Requirements:

If the action is an instance of `DoubleConsumer` then it is cast to `DoubleConsumer` and passed to `forEachRemaining(DoubleConsumer)`; otherwise the action is adapted to an instance of `DoubleConsumer`, by boxing the argument of `DoubleConsumer`, and then passed to `forEachRemaining(DoubleConsumer)`.

Parameters:

`action` - The action to be performed for each element

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.